

第5章 中間コード生成

5.1 概要

5.1.1 逆ポーランド記法

逆ポーランド記法は引数の数が固定の演算を後置形式で記載する。計算に用いる値をメモリの値を `push` 命令で参照することによって計算することができ、算術式の間接コードとして古くから用いられている。逆ポーランド記法からより低レベルの機械語（バイナリコード）へ変換することは容易である。演算の引数となる変数を *ID* で参照するとき、`push ID` でその値をスタックに積む、演算を適用するとき、*n* 引数の演算では、*n* 個の値をスタックから `pop` して演算を適用し、その結果をスタックに `push` する。例えば、 $x+y*z$ という入力に対しては、`push x, push y, push z, *, +` という命令列によってスタックの先頭に値の計算が積まれる (図 5.1)。

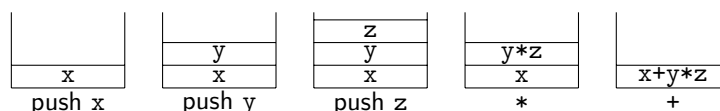


図 5.1: 逆ポーランド記法の計算

実際には構文木の構成において、 x, y, z はそれぞれトークンとして *ID* として扱われるが、実際のコード生成においては、変数名を必要とする。字句解析においてもトークンの他に実際の識別子の記号列が保存されている。(lex における `yytext` など)

5.1.2 抽象構文木

抽象構文木 (abstract syntax tree) は、構文解析木をコード生成に必要な部分のみを残して変形した木構造である。コード生成において、構文解析木に現れる非終端記号の情報は必要でない場合が多く、生成規則の持つ意味をノードに記述した方がコード生成には都合が良い。木構造がコードの意味を直接的に表現していると考えられることから、抽象構文木が中間コードであるとみなすことができる。例えば、代入式 $w = x + y * z$ の抽象構文木は図 5.2 のようになる。

5.1.3 3 番地コード

3 番地コード (3-address code) は、命令を一般的に $C = A \theta B$ の形に限定して動作を記述する。ここで、 A, B, C はメモリ番地であり、 A のメモリの内容と B のメモリの内容に θ の演算を適用し

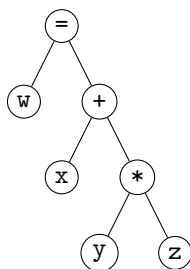


図 5.2: 抽象構文木の例

て、結果を C の番地に格納することを表す。これ以外に単項演算、条件付きジャンプ命令などを用いることによって一般的な計算を単純な命令の組み合わせで記述することができる。各命令にはラベル L をつけることができ、ジャンプ命令の飛び先とすることができる。3 番地コードを機械語に変換することは容易であり、3 番地コードによって最適化を行うことでハードウェアに依存しないコード最適化を適用することができる。

```

int fib(int x) {
    if (x=0) {
        return 0;
    } else if (x=1) {
        return 1;
    } else {
        return fib(x-2)+fib(x-1);
    }
}

print fib(4);

```

```

define @fib(x) {
    t1 := x
    t2 := (x=0)
    if not t1 goto l1
    return 0
l1: t3 := (x=1)
    if not t2 goto l2
    return 1
l2: t4 := t1 - 2
    t5 := call fib(t4)
    t6 := t1 - 1
    t7 := call fib(t6)
    t8 := t5 + t7
    return t8
}

define @main() {
    t1 := 4
    t2 := call fib(t1)
    print t2
}

```

図 5.3: 3 番地コードへの変換例

図 5.3 に変換例を示す。ここでは、関数呼び出しとして @関数名 という形をとっている。

LLVM はコンパイラ研究でよく用いられる研究基盤である。3 番地コードを基本として中間コードとして LLVM-IR が提案されている。LLVM-IR では静的単一代入形式でデータフローが式の解析によって得られ、最適化を容易にしている。図 5.4 に変換例を示す。

```

; ModuleID = 'main.c'
source_filename = "main.c"

@.str = private unnamed_addr @backslash\
    constant [4 x i8] c"%d\0A\00", align 1

; Function Attrs: noinline nounwind optnone uwtable
define i32 @fib(i32) #0 {
    %2 = alloca i32, align 4
    store i32 %0, i32* %2, align 4
    %3 = load i32, i32* %2, align 4
    %4 = icmp eq i32 %3, 0
    br i1 %4, label %5, label %6

5:
    ret i32 0

6:
    %7 = icmp eq i32 %3, 1
    br i1 %7, label %8, label %9

8:
    ret i32 1

9:
    %10 = load i32, i32* %2, align 4
    %11 = sub nsw i32 %10, 2
    %12 = call i32 @fib(i32 %11)
    %13 = load i32, i32* %2, align 4
    %14 = sub nsw i32 %13, 1
    %15 = call i32 @fib(i32 %14)
    %16 = add i32 %12, %15
    ret i32 %16
}

```

図 5.4: LLVM-IR への変換例

$\%n$ は、レジスタ n を表す。中間コードでは、レジスタは任意数あるとする。LLVM-IR は、代入ごとにレジスタの番号を変えることによって静的単一代入 (static single assignment, SSA) を実現する。LLVM-IR は実用的な中間コードであり、基本的なデータ型やビット長などが詳細に記述される。以下に主な命令を示す。(「きつねさんにもわかる LLVM」より引用)

alloca	result=alloca(type) [, (type) (NumElements)] [, align (alignment)]	メモリ確保命令 type × NumElements の領域を関数のスタックフレーム上に確保
load	result=load [volatile] (type)* (pointer) [, align (alignment)] [, !nontemporal! (index)] [, !invariant.load ! (index)] result=load atomic [volatile] (type)* (pointer) (singlethread) (ordering) !index=! i32 1 align (alignment)	ロード命令 pointer が指すメモリアドレスから値を読み出す volatile が指定されている場合、最適化は load 命令の実行順や実行数をほかの volatile な命令に変更することができない
store	store [volatile] (type) (value) (type)* (pointer) [, align (alignment)] [, !nontemporal ! (index)] store atomic [volatile] (type) (value) (type)* (pointer) [singlethread] (ordering) align (alignment)	ストア命令 pointer が指すメモリアドレスに値を格納する volatile が指定されている場合、最適化は store 命令の実行順や実行数をほかの volatile な命令に変更することができない
getelementptr	result = getelemntptr (ptypel)* (ptrval) {, (type) (idx) }* result = getelementptr inbounds (ptypel)* (ptrval) {, (type) (idx) }* result = getelementptr (ptr vector), (vector index type) (idx)	要素アドレス取得命令 合成型内の要素のアドレスを計算する ptrval が指すアドレスをベースとして idx で指定したインデックスの要素アドレスを計算する

図 5.5: LLVMIR(メモリアクセス)

add	result = add (type) (op1) (op2) result = add nuw (type) (op1) (op2) result = add nsw (type) (op1) (op2) result = add nuw nsw (type) (op1) (op2)	加算命令 op1 と op2 を加算して結果を返す。op1, op2 は整数か整数のベクタ。 nuw = No Unsigned Wrap, nsw = No Signed Wrap/ オーバーフローが発生したときに結果が未定義
------------	--	---

同様な演算命令 fadd, sub, fsub, mul, fmul, udiv, sdiv, fdiv, urem, srem, frem(剰余)

図 5.6: LLVMIR(演算命令)

ret	ret (type) (value) ret void	return 命令 関数の呼び出し元へ制御を戻す value が指定されていれば value を戻す
br	br il (cond), label (iftrue), label (iffalse) br label (dest)	分岐命令 cond の true/false に応じて分岐先が変わる cond が指定されない場合はラベル dest に無条件分岐する
icmp	result=icmp (cond) (type) (op1) (op2)	比較命令 op1, op2 を cond で指定した条件で比較し、真偽を返す。 op1, op2 は整数、ポインタか整数のベクトル
call	result = [tail] call [cconv] [ret attrs] (type) [(fnty)*] (fnptrval) ((function args))[fn attrs]	関数呼び出し命令 関数 fnptrval を function args を引数として呼び出す。

図 5.7: LLVMIR(制御命令)

eq	equal 2つの引数が同値の場合 true, それ以外 false
ne	equal の否定
ugt	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きい場合に true, それ以外 false
uge	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きいか同じ場合に true, それ以外 false
ult	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さい場合に true, それ以外 false
ule	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さいか同じ場合に true, それ以外 false
sgt	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きい場合に true, それ以外 false
sge	2つの引数を unsigned value として扱って第一引数が第二引数よりも大きいか同じ場合に true, それ以外 false
slt	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さい場合に true, それ以外 false
sle	2つの引数を unsigned value として扱って第一引数が第二引数よりも小さいか同じ場合に true, それ以外 false

図 5.8: LLVMIR(比較命令)

5.2 実行環境

プログラムでは局所変数コードを実行する場合には,

5.3 記号表

5.4 型チェック